

Week 4 Plan; Sequential Agent

```
plan Shift+Tab to manual
```

```
> █ Type your message or @path/to/file
```

```
workspace (/directory)
```

Subprocess Cheatsheet

Claude

[Claude_CLI_Subprocess_API.md](#)

Codex

[Codex_CLI_Subprocess_API.md](#)

Gemini

[Gemini_CLI_Subprocess_API.md](#)

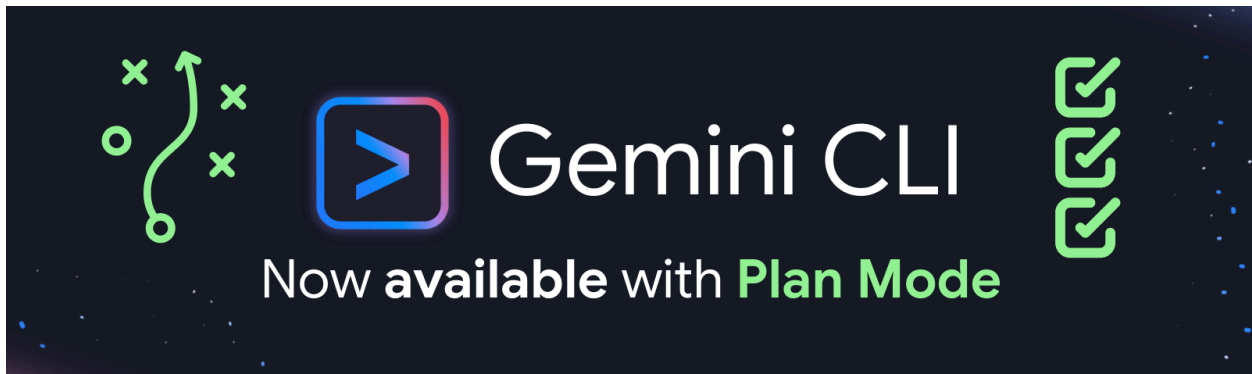
Heavy User (Claude + Codex + Gemini)

[Agent_CLI_Subprocess_API.md](#)

[API 레퍼런스 압축파일.zip](#)

팁: Claude와 Codex를 동시에 사용하는 경우 Claude의 Skill에 Codex를 Implementation 담당으로 등록, Codex의 Skill에 Claude를 Review 담당으로 등록하는 것으로 CLI 레벨에서 간단한 에이전트 코딩 체계를 확보할 수 있습니다.

Plan Mode



- Agent CLI에서 **계획만 세우고, 코드는 건드리지 않는** 모드
- 진입하면 read-only 도구만 활성화
- 파일 읽기, 검색, 웹 조회는 되지만 파일 수정·생성·삭제·명령 실행은 차단
- 산출물은 마크다운 파일로 저장됨 (At Memory space)
- 사용자가 계획을 검토하고 승인하면 그때 구현 모드로 전환

```
> Codex_CLI_Subprocess_API.md를 읽고, 최대한 간단한 방식으로 Subprocess 기반 대화형 GUI 환경을 구성할 방안에 대해 알려주세요.
```

```
gpt-5.4 medium fast · 100% left · C:\Termis\T3\coversation
```

```
Plan mode
```

Assumptions

- 플랫폼은 Windows이며 `codex.cmd` 가 PATH 에 있다.
- 최소 구현 목표는 “로컬에서 Codex와 대화 가능한 GUI” 이지, 범용 채팅 제품이 아니다.
- 기본 권한값은 단순화를 위해 `-a never`, `--sandbox workspace-write` 로 둔다.
- 시스템 프롬프트 커스터마이징이 필요하다면 2단계로 `model_instructions_file` 만 추가하고, 1단계에서는 제외한다.
- 가장 단순한 선택으로 Tkinter desktop app 을 채택한다. 웹 GUI보다 의존성과 운영 복잡도가 낮다.

Implement this plan?

- > 1. Yes, implement this plan Switch to Default and start coding.
- 2. No, stay in Plan mode Continue planning with the model.

Press enter to confirm or esc to go back

- 파이썬 기반으로 간단한 프로토타입을 요청하면 Tkinter로 개발을 시도하려는 특징이 있음, Tkinter는 추가 라이브러리가 필요가 없고 Fault가 발생할 확률은 낮지만 성능 이슈가 있음.

이상적인 Plan Mode의 형태 (Claude)

□ GUI 선택

GUI 프레임워크로 어떤 것을 사용하시겠습니까?

- 1. Gradio (권장)
- 2. Tkinter
- > 3. 웹 (Flask + HTML/JS)

구조:

`app.py` - Flask 서버 + SSE endpoint

11 lines hidden

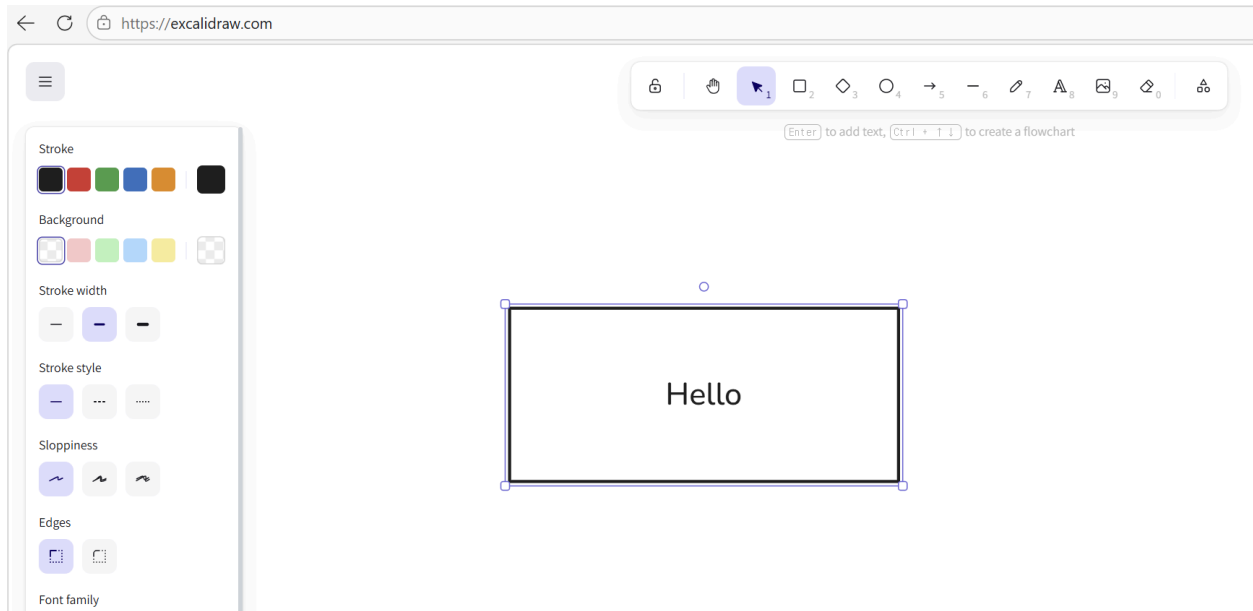
Notes: press n to add notes

Chat about this

Skip interview and plan immediately

- 대신 이거 보여주기 위해서 토큰을 펍펍 쓰기 때문에 비효율적임

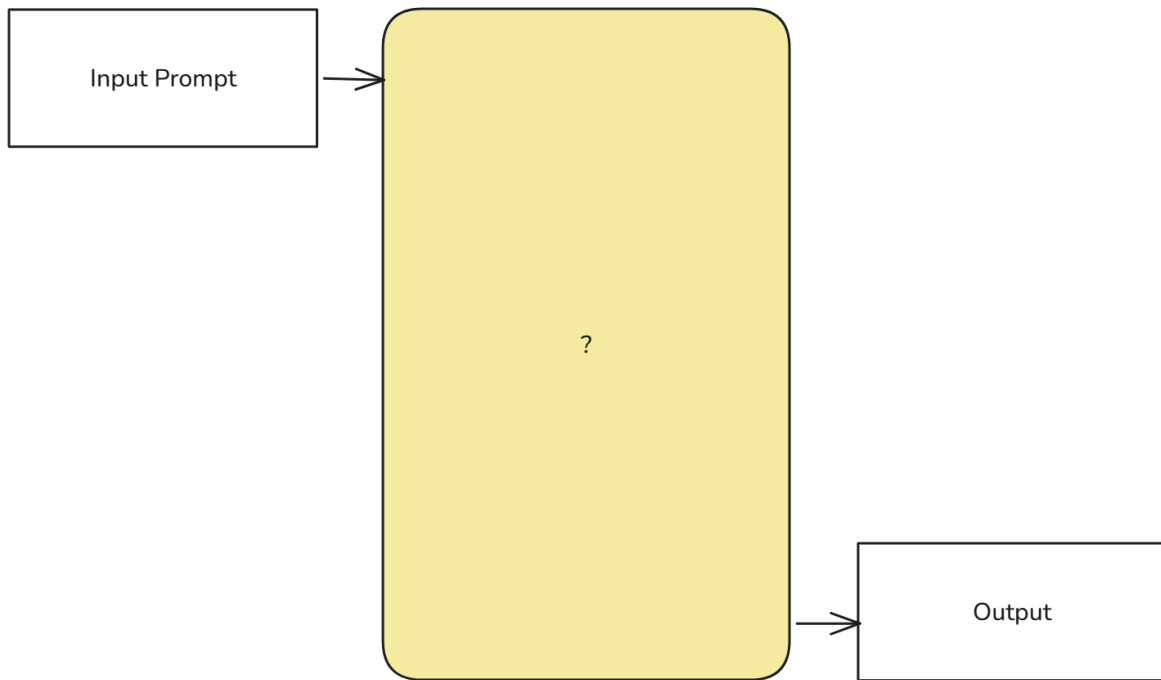
System Design



Excalidraw Whiteboard

Goal

- 프롬프트를 입력하면 Plan, Todo를 생성하는 자동화 파이프라인을 설계합니다.



Task 1

- AI와 논의하여 Plan Mode의 구조와 산출물을 파악합니다.
- Excalidraw를 이용하여 러프하게 파이프라인 구조를 정의합니다.

Task 2

```

YOLO Ctrl+Y 1 GEMINI.md file

* @C:\Users\meno9\Pictures\Screenshots\스크린샷 2026-03-25 145034.png"
  @C:\Termis\T3\coverstation\Codex_CLI_Subprocess_API.md 두 가지 파일을 확인하여 subprocess를 이용해 사용자의 Prompt
  Input Request를 Planning -> Review -> Revised Planning -> TODO 도출 -> Markdown Output 하는 스크립트를 작성하고,
  테스트 실행해주세요.

workspace (/directory) sandbox /model
C:\Termis\T3\coverstation no sandbox Auto (Gemini 3)
  
```

- 생성한 파이프라인 구조 image와 API Document를 drag & drop을 이용해 CLI에게 전달하고 실행하도록 명령합니다.
- “파이프라인 스크립트의 명세를 재사용 가능하도록 구조화한 문서를 작성하고 markdown 파일로 저장해주세요” 와 같이 요청하여 Pipeline Structure를 저장합니다.

Task 3

- 각 Round 또는 개별 Agent 마다 이번 작업내용을 기록하는 Handoff 파일을 저장할 수 있도록 파일 프라인 구조를 개선합니다. (예시: workspace/00_planning.md, 01_review.md, ... , 04_final_plan_report.md)

실습 과제

강의자료를 참고하여 Pipeline Structure의 산출물(Hand-off markdowns)를 압축파일로 제출합니다.

주요 목표 : **subprocess 기반 AI 챗봇의 구현을 위한 설계 문서 생성**

예시 :

1. Plan 문서 (구현 계획)
2. Plan 문서에 대한 리뷰 문서 (구현 계획에 대한 평가 문서)
3. Revised Plan 문서 (수정된 구현 계획 문서)
4. TODO 문서 (구현을 위한 체크리스트 문서)

Pipeline 스크립트 만들어서 매 단계 혹은 각 에이전트마다 Hand-off 파일을 저장하도록 해달라고 하시면 됩니다.

과제는 문서의 품질을 고려하지 않습니다.

그래도 이왕이면 산출물 재사용 가능하게 작성해보세요. (구현까지 해보시면 아주 좋음)

Week 4 보강자료